ゼミ発表資料（2026-06-09 週）

今週の要点

今週は **問題セット（実験の土俵）の確定** がメイン。あわせて計算時間の最適化と実験ハーネスの再整備を行った。

1. **問題セットの確定** — 「難易度をどう作るか」を決着させた。結論は **難易度軸＝外乱の "位置"**（遅延量でも問題規模でもない）。あわせて積年の `reschedule_time` バグを発見・修正し、安定性目的を $\beta=0$（重みなし順列偏差）へ変更。最終的に **6 シナリオ** を生成・非縮退確認まで完了。
2. **計算時間の最適化** — 安定性計算の O(n) 化・差分化に加え、PR を **random-walk relinking** に切り替えて decode 回数を O(diffs²)→O(diffs) に削減（3〜13 倍高速）。検討したが不採用の選択肢も整理。
3. **実験ハーネス v3 の整備** — 確定した 6 シナリオ・7 手法で回す `core_comparison_v3` を作成（本走待ち）。

---

1. 問題セットの確定 ＝ 今週の本丸

1.1 出発点：「外乱の大小」をどう作るかが詰まっていた

これまで「外乱小→S_p 近傍→ILS 有利／外乱大→遠い→Memetic・PR 有利」という仮説で実験を組んできた。 ところが **外乱の大小を "単一遅延の大きさ δ" で作ろうとすると、手法間の差が出ない** ことが分かってきた。 そこで「そもそも難易度をどのレバーで作るのか」を根本から再検討した。

1.2 難易度の物差し ＝ headroom

難易度（＝アルゴリズムが差を出せる土俵があるか）を測る指標として **headroom** を導入した。

> **headroom** ＝ `RSR_makespan − 最適化後makespan`（元の順序のまま右シフトするだけの RSR に対し、順序を並べ替えて makespan をどれだけ縮められるか）

- `headroom = 0` → 右シフトが既に最適。**全手法が同じ解に潰れる（縮退・区別不能）**
- `headroom 大` → 並べ替えに価値があり、良い手法ほど縮む → **手法間の差が出る**

縮退したシナリオで比較しても全手法横並びになるだけなので、**headroom > 0 を必ず確認** してから実験に使う、という運用にした。 このために診断ツール `tools/measure_headroom.py` を作成（`--scenario` で既存シナリオの測定、`--scan` で全クリティカル op を振った headroom 分布）。

1.3 結論：難易度は「遅延量」でも「問題規模」でもなく「外乱の位置」で作る

3 つのレバーを実測で比較した結果が以下。

レバー	headroom の差	採否
遅延量 δ	**出ない**（δ に不変）	✗
問題セット	バラつくが制御不能（mt10:29 / la21:24 / la40:72）	△ 一般性用。狙った難易度は作れない
外乱の位置	**大きく動く**（早い外乱→大 / 遅い外乱→小）	✅ **採用**

なぜ遅延量 δ では難易度が作れないのか（理論）:

> クリティカル路の遅延は、再スケ問題に「**全体を一定量だけ右にずらす**」効果（定数の right-shift オフセット）しか持たない。 RSR も最適解も同じ δ を吸収するので、その差＝headroom は δ に不変。「時間軸で S_p から遠い」と「順序空間で最適解が遠い」が、単純遅延では **デカップル** してしまう。複数遅延も臨界路上では一様シフトの重ね合わせで同じ。非フレキシブル JSP では作業遅延と機械故障は同一ファミリー（待つ＋並べ替えるのみ）なので故障でも同結論。

実測でも δ・遅延箇所数は headroom を動かさなかった：

シナリオ	外乱	headroom
mt10 delay60 → delay90	単一 δ +50%	29 → 29（不変）
la36 単一 → multi3 → multi5	+120 → +242	26 → 26 → 26（不変）

→ 文献とも整合（Zhang 2013 は到着率・新ジョブ数で大小を作る／Kundakcı 2016 は問題規模をスケール軸にする。どちらも「単一遅延を何分にするか」ではない）。

外乱の "位置" が効く理由:

外乱の位置は「スケジュールのどれだけを再最適化するか（reschedule horizon）」を直接決める。

```
早い外乱 → 再スケ範囲 大 → 最適解が S_p から遠い → headroom 大 → ILS の近傍前提が崩れ Memetic/PR が有利
遅い外乱 → 再スケ範囲 小 → S_p 近傍              → headroom 小 → ILS で十分
```

これは当初の仮説を「δ の代わりに位置で忠実に実装」したもの。 しかも位置は **同じ問題の中で** 難易度を振れる（問題規模では別問題になってしまい不可能）。

1.4 ただし位置は horizon を綺麗に制御するが headroom は op 依存（実測, la36）

pos	reschedule horizon	headroom
0.00	198	59
0.17	164	**62** ← 大の候補
0.26	142	**0（縮退）**
0.40	114	**0（縮退）**
0.52	75	64
0.72	29	**29** ← 小の候補
0.78	14	**0（縮退）**
0.87	3	0（縮退）

- **reschedule horizon は位置に綺麗に比例**（早 198 → 遅 3）＝「近傍/遠い」は位置で確実に制御できる。
- **headroom は位置に綺麗には比例しない**（0〜64 でガタつく。同じ大 horizon でも 0.26/0.40 は縮退）。op ごとの局所競合構造に依存するため。
- ∴ **位置を決めた上で `measure_headroom` で headroom>0 の op を選ぶ**（機械的に位置だけで選ぶと縮退を引く）。cherry-pick ではなく「非縮退の確認」。

1.5 副産物：`reschedule_time` の +1 バグを発見・修正（mt10 の縮退の真犯人）

headroom を測る過程で、**mt10 が headroom 0 に潰れて見えていた原因が外乱設計ではなくバグ** だと判明した。

> `reschedule_time = 遅延 op の right-shift 後の終了時刻`（**+1 しない**）。 gantt_chart_operation.py で `latest_end + 1` → `latest_end` に修正。

- 遅延 op の **直後工程** は「遅延 op 終了」ちょうどに開始する。+1 すると `start < reschedule_time` 判定でこの直後工程まで凍結され、**外乱に直撃された一番リスケすべき工程を最適化対象から外していた。**
- これが mt10 を headroom 0 に見せていた原因（5/26 の複数遅延書換 commit で混入したリグレッション）。
- **修正後: mt10_delay60 headroom 0 → 29**（la21=24 / la36=26 は不変）。

→ mt10 は「縮退する問題」ではなく、本来は headroom を持つ普通の問題だった。設計判断をバグの上で誤りかけていたので、これは大きい。

1.6 安定性目的を $\beta=0$（重みなし順列偏差）へ変更

- **安定性 = S_p からの順列偏差（重みなし, $\beta=0$）**：各機械ごとに `Σ |init_pos − cur_pos|` を合計。
- 卒論では $\beta=1.25$ の重み付き偏差だったが、修論では **$\beta=0$（重みなし）** に変更。
  - 理由: makespan と直交し、近傍演算子（順序の入れ替え）と整合的なため。
  - pilot で $\beta=0$ でも memetic+PR > LS の HV 優位（large）・フロント非潰れを確認済み。

1.7 確定した 6 シナリオ（生成・非縮退確認済み）

シナリオ	問題	サイズ	役割	位置	horizon	headroom
`mt10_delay60`	mt10	10×10	一般性	中	70	29
`la21_delay147`	la21	15×10	一般性	中	53	24
`la40_delay148`	la40	15×15	一般性（同サイズ異構造）	中	73	72
`la36_large`	la36	15×15	**難易度 大（遠い）**	0.17	164	62
`la36_small`	la36	15×15	**難易度 小（近傍）**	0.72	29	29
`ta21_delay97`	ta21	20×20	**大規模 showcase**	0.55	128	57

設計の骨子：

- **位置軸（難易度ラダー）は la36 で実施** — 大（早い 0.17）／小（遅い 0.72）の 2 水準を §1.4 の実測で選定。
- **一般性は mt10/la21/la40** — それぞれ中間位置の単一外乱で非縮退を確認。
- **大規模 showcase に Taillard ta21（20×20）を追加** — la 以外の family を入れて一般性も補強。`problems/ta21.txt` を JSPLIB から追加。
- **遅延量は対象 op の PT × 1.0（処理時間を倍に）で全シナリオ共通固定** — slack を超えてリスケを誘発する最小限で十分。headroom は δ 不変なので大きくする意味がない。
- **S_p（初期スケジュール）は GA 500 世代** — 最適化しすぎない。締めすぎると再スケ余地（headroom）が減る。Memetic は最適寄りで余地を潰すため不可。

旧シナリオ（mt10_delay90 / la36_delay148 / la36・la40 の multi3・multi5）は削除済み。

---

2. 計算時間の最適化

確定した問題セット（特に大規模 ta21・大外乱 la36_large）を 7 手法 × 11 重み × 30 trial で回すには計算量が支配的になる。本走前に律速箇所を潰した。**いずれも出力厳密不変を検証済み**。

2.1 安定性計算の高速化（GA/ILS/memetic 全 evaluate に効く）

- **O(n) 化**: `_rank_deviation` のループ内 `.index()`（O(L²)）を位置辞書で O(L) に。長い外乱（la36_large）では evaluate のかなりを占めていた。
- **差分化（O(1)）**：「親 + 1 スワップ」型の近傍は、親の安定性を 1 回計算し、各候補は動く 2 位置の項だけ O(1) で更新（semi-active 限定、active はフル評価にフォールバック）。
  - **PR**: 候補 1 個あたり evaluate を ~19% 削減（la36_large 540→439µs）。
  - **N5 局所探索**: Taillard スクリーニング 1 巡の安定性コストを **~80% 削減**（la36_large 638→129µs）。**ILS・memetic の局所探索すべてに効く**。

2.2 PR を random-walk relinking へ（本丸の高速化）

プロファイル（la36_large）の結果、evaluate の **約 73% が build_gantt（デコード）** だった。安定性は最適化済みなので、残りは **decode の "回数" を減らす** 問題。

- **原因**: best-selection の PR は各ステップで残り不一致数ぶんの全候補をフル decode して最良を選ぶ → 1 回の PR で **O(diffs²) 回の decode**。diffs は外乱の reschedule 範囲に比例するので、大外乱ほど高コスト（旧 la36_delay148 で ~3000s/試行）。
- **対策: random-walk relinking を採用（既定）** — 各ステップで実行可能な direct swap を **1 つランダムに選ぶ**（TS/PR, Peng et al. 2015 流）。decode が **O(diffs²) → O(diffs)** に激減。
- **効果**（la36_large, ngen=100, n=10）: best-selection 比 **約 3〜13 倍高速**、UEA HV は **有意差なし**（**Wilcoxon p=0.72**）・高安定ゾーンも同等。
- → memetic・ILS とも既定を `'random'` に切替。best-selection は sweep 比較用に `'best'` で残す。
- 副次効果: random-walk で目的を達成したため、検討していた **Taillard 下界の枝刈り／未改変個体スキップは不要** になった（Taillard 推定は N5 隣接 swap 専用で任意 transposition に使えず実装困難だった）。

2.3 検討したが不採用（random-walk 確定後の追加検討）

「同じ問題を何度も検討し直さない」ために doc/PR_computing_reducing.md に所見を集約。

検討項目	結果	判断
FI（first-improvement）	BI と同品質だが CPU ~25% 遅い（S_p 方向は改善 swap がほぼ出ず実質全候補スキャン＋着地が悪く LS が重い）	✗ 時短目的では不採用
始点・終点除外（return_intermediate）	HV 有意差なし（p=0.50）だが include 比 ~28% 遅い（悪い中間解を返しキック後 LS が重い）	✗ include 維持
top-3 LS（スカラー上位 3 点を各 LS）	HV 有意差なし（p=0.50）だが約 2 倍遅い	✗ k=1 維持

まとめ: random / best / include / exclude / top-3 は互いに HV 有意差なし。**random k=1 + include が「同品質で最速」** なので既定として確定。（経路の途中打ち切り `escape_infeasible` は小外乱で発火 0/50 と稀

だが、起きたとき確実に経路を繋ぐ突破ロジックを追加し memetic で有効化。）

---

3. 実験ハーネス v3（`core_comparison_v3`）

確定した 6 シナリオ・7 手法で回すスクリプトを整備（本走待ち）。

- **構成**: (problem × weight × method × trial) をフラット化し `ProcessPoolExecutor` で並列。**1 ファイル = 1 run** で保存するので、途中停止・再開・部分実行・重み分割統合に対応（既存ファイルはスキップ＝resume）。
- **対象**: 6 シナリオ × 7 手法（GA / ILS-baseline / ILS+repair / ILS+PR / Memetic-LS / Memetic+repair / Memetic+PR）× 11 重み × 30 trial ≈ **14,000 run**。
- ⚠ ta21（20×20）は 1 run が長く支配的になる見込み。探索段階は重み点/trial を絞り、確定後の最終ランのみ精密に、という **staging** を推奨。

---

4. 次のステップ

- **コア実験 v3 の本走**（6 シナリオ × 7 手法 × 11 重み × 30 trial、staging 推奨）。
- la36 位置軸（大 0.17 / 小 0.72）で「位置→headroom→手法間の差」の仮説を検証 — **早い外乱で Memetic/PR が ILS に勝てるか** が最大の見どころ。
- 大規模 ta21 でのスケール挙動の確認（ILS の速度優位 vs Memetic の質）。
- （任意）一般性 3 問題の遅延量を PT×1.0 に揃えるか判断（δ 不変なので必須ではない）。

---

5. 議論したいこと

- **難易度軸を「外乱の位置」に一本化した判断の妥当性** — 遅延量・問題規模を難易度レバーから外し、位置（reschedule horizon）に絞ったことを論文の主軸に据えてよいか。
- **headroom を「土俵の有無」の指標として論文に出す妥当性** — RSR 比の縮め幅という定義で説得力があるか。
- **mt10 縮退がバグ由来だった件の扱い** — 過去の「縮退する」という記述を撤回し、6 問題すべてを非縮退で揃えた経緯をどこまで本文に書くか。
- 安定性 β=0（重みなし順列偏差）への変更を、卒論からの一貫性も含めてどう位置付けるか。

付録: 用語ミニ辞典

用語	意味
headroom	`RSR_makespan − 最適化後makespan`。並べ替えで縮められる余地＝「手法差が出る土俵」の指標。0 なら全手法縮退
RSR (Right-Shift Rescheduling)	元の順序を保ったまま遅延ぶん右にずらすだけの素朴な再スケ
reschedule horizon	外乱以降で再最適化対象になるスケジュールの範囲。外乱が早いほど大きい
reschedule_time	リスケ対象/凍結を分ける境界時刻。遅延 op の右シフト後終了時刻（+1 しない）
縮退（degeneracy）	headroom=0 で全手法が同一解に潰れ、区別不能になる状態
順列偏差（β=0）	各機械で ` Σ
random-walk relinking	PR の各ステップで実行可能 direct swap を 1 つランダム選択。decode を O(diffs²)→O(diffs) に
direct swap	PR のムーブ。離れた位置のジョブを 1 ステップで交換し 1 不一致位置を解消
S_p	外乱前の予測（初期）スケジュール。安定性の基準
diffs	S_{cur} と S_p の不一致位置数。PR の経路長＝外乱規模に比例